

LAB 1.1(Creating Dataset)

```
import numpy as np

import pandas as pd

print(np.__version__)

print(pd.__version__)

a=np.array([1,2,3,4,5,6])

print(a)

mean=np.mean(a)

print(mean)

data={

    'rollno':[1,2,3,4,5,6,7,8,9,10],

    'Age':[20,21,np.nan,22,np.nan,22,24,21,22,23],

    'Gender':['M','M',np.nan,np.nan,'F','M','M','F','M','F'],

    'Depaerment':['CSE','CSBS','AIML','CSBS','CSE','ME','CSE','CSDS','CSE','CSE']

}

df=pd.DataFrame(data)

print(df)

df.info()

df.head()

df.tail()

df.shape

df.count()

df.mean(numeric_only=True)

df['Age'].mean()

df.median(numeric_only=True)

df['Age'].median()

df.mode()

df.std(numeric_only=True)

df.describe()

df.isna()

df.isna().sum()
```

```
df.fillna({'Age': df['Age'].mean()}, inplace=True)
```

```
df.fillna({'Gender': df['Gender'].mode()[0]}, inplace=True)
```

LAB – 1.2 (Import Dataset - Diabetes)

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
df=pd.read_csv("diabetes.csv")
```

```
df
```

```
df.info()
```

```
df.describe()
```

```
df.isna().sum()
```

```
df.duplicated().sum()
```

```
import seaborn as sns
```

```
sns.boxplot(df['Insulin'])
```

```
for col in ['Glucose','BloodPressure','SkinThickness','Insulin']:
```

```
    q1 = df[col].quantile(0.25) # 25th percentile
```

```
    q3 = df[col].quantile(0.75) # 75th percentile
```

```
    iqr = q3 - q1           # Interquartile range
```

```
    lower = q1 - 1.5 * iqr    # Lower bound
```

```
    upper = q3 + 1.5 * iqr    # Upper bound
```

```
    df = df[(df[col] >= lower) & (df[col] <= upper)]
```

```
df.shape
```

```
#KDE plot for Insulin
```

```
sns.kdeplot(df['Insulin'], fill=True, color="Black")
```

```
plt.title("KDE Plot of Insulin")
```

```
plt.show()
```

```
for col in df.columns:
```

```
    if df[col].dtype != 'object': #only numeric value
```

```

    print(col, "=", df[col].skew())
df['Insulin_log']=np.log1p(df['Insulin'])
df['Age_log']=np.log1p(df['Age'])
df['DPF_log']=np.log1p(df['DiabetesPedigreeFunction'])
sns.kdeplot(df['Insulin_log'], fill=True, color="Black")
plt.title("KDE Plot of Insulin")
plt.show()
for col in df.columns:
    if df[col].dtype != 'object': #only numeric value
        print(col, "=", df[col].skew())
corr=df.corr()
sns.heatmap(df.corr(),annot=True,fmt='.2f')
plt.show()

```

LAB – 2 (Visualization)

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data={
    'Student':['A','B','C','D','E','F','G','H'],
    'math':[85,78,56,98,52,48,56,89],
    'science':[75,89,41,68,55,49,48,75],
    'english':[65,97,85,65,36,89,56,84],
    'sports':[66,85,42,96,55,36,55,44]
}

df=pd.DataFrame(data)

print(df)

sns.scatterplot(x='math',y='science',data=df, hue='Student',palette='Set1')

plt.title('Scatter Plot: Math vs Science Score')

plt.show()

```

```

avg_scores=df[['math','science','english','sports']].mean()

sns.lineplot(x=avg_scores.index,y=avg_scores.values,marker='*')

plt.title("Line Plot: Average Scores per score")

plt.show()

plt.pie(avg_scores,labels=avg_scores.index,autopct="%1.1f%%",colors=['Red','Green','lightblue','lightpink'])

plt.title("Pie Chart: Subject Score Distribution")

plt.show()

sns.boxplot(y='english', data=df, color='skyblue')

plt.title("Box plot: English Score")

plt.show()

sns.boxplot(data=df[['math','science','english','sports']],palette='Set2')

plt.title("Box Plot: Scores across subject")

plt.show()

sns.histplot(df["science"],bins=5,kde=True,color="green")

plt.title("Histogram: Science Score Distribution")

plt.show()

corr=df.drop("Student",axis=1).corr()

sns.heatmap(corr,annot=True,cmap="coolwarm")

plt.title("Heatmap: Subject Score Correlation")

plt.show()

#Load iris dataset directly from seaborn

iris=sns.load_dataset("iris")

print(iris.head())

sns.scatterplot(x='sepal_length',y='petal_length',data=df, hue='species',palette='Set1')

plt.title('Scatter Plot: sepal_length vs petal_length')

plt.show()

avg_scores=df[['sepal_length','sepal_width','petal_length','petal_width']].mean()

sns.lineplot(x=avg_scores.index,y=avg_scores.values,marker='*')

plt.title("Line Plot: Average Scores per score")

plt.show()

```

```

plt.pie(avg_scores, labels=avg_scores.index, autopct="%1.1f%%", colors=['Red', 'Green', 'lightblue', 'lightpink'])

plt.title("Pie Chart: Subject Score Distribution")

plt.show()

sns.boxplot(y='sepal_length', data=df, color='skyblue')

plt.title("Box plot: sepal_length")

plt.show()

sns.boxplot(data=df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']], palette='Set2')

plt.title("Box Plot: Across all data")

plt.show()

sns.histplot(df["sepal_length"], bins=5, kde=True, color="green")

plt.title("Histogram: sepal_length Score Distribution")

plt.show()

corr=df.drop("species",axis=1).corr()

sns.heatmap(corr,annot=True,cmap="coolwarm")

plt.title("Heatmap: iris dataset Correlation")

plt.show()

sns.pairplot(iris, hue="species", diag_kind="kde")

plt.show()

```

LAB – 3 (Regression - AreaPriceDataset.xlsx)

```

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

df=pd.read_excel("AreaPriceDataset.xlsx")

df.isnull().sum()

plt.plot(df['Area'],df['Price'])

plt.xlabel("Area")

plt.ylabel("Price")

```

```

plt.grid(True)

plt.show()

sns.kdeplot(df['Area'])

sns.kdeplot(df['Price'])

plt.scatter(df['Area'],df['Price'])

from sklearn.linear_model import LinearRegression

X=df[['Area']]

y=df[['Price']]

X_train, X_test, y_train, y_test, = train_test_split(X,y,test_size=0.2, random_state=42)

model=LinearRegression()

y_pred=model.fit(X_train,y_train)

model=LinearRegression()

y_pred.predict(X_test)

y_prediction= y_pred.predict(X)

print(y_pred.coef_)

print(y_pred.intercept_)

from sklearn.metrics import mean_squared_error, r2_score

mean_squared_error(y,y_prediction)

r2_score(y,y_prediction)*100

plt.plot(X,y)

plt.plot(X,y_prediction)

```

LAB – 3 (Regression - House_Rent_Dataset.csv)

```

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

from sklearn.preprocessing import StandardScaler

```

```
df=pd.read_csv("House_Rent_Dataset.csv")
df=df[['Size','Rent']]
df
df.isnull().sum()
df.shape
df.describe()
df.info()
df.isnull().sum()
df.duplicated().sum()
df.drop_duplicates(inplace=True)
df.shape
plt.plot(df['Size'],df['Rent'])
plt.xlabel("Size")
plt.ylabel("Rent")
plt.grid(True)
plt.show()
sns.boxplot(df['Size'])
q1 = df['Size'].quantile(0.25) # 25th percentile
q3 = df['Size'].quantile(0.75) # 75th percentile
iqr = q3 - q1                # Interquartile range

lower = q1 - 1.5 * iqr      # Lower bound
upper = q3 + 1.5 * iqr     # Upper bound

df = df[(df['Size'] >= lower) & (df['Size'] <= upper)]
sns.boxplot(df['Size'])
sns.boxplot(df['Rent'])
q1 = df['Rent'].quantile(0.25) # 25th percentile
q3 = df['Rent'].quantile(0.75) # 75th percentile
iqr = q3 - q1                # Interquartile range
```

```
lower = q1 - 1.5 * iqr    # Lower bound
upper = q3 + 1.5 * iqr    # Upper bound
```

```
df = df[(df['Rent'] >= lower) & (df['Rent'] <= upper)]
sns.boxplot(df['Rent'])
sns.kdeplot(df['Size'])
sns.kdeplot(df['Rent'])
plt.scatter(df['Size'],df['Rent'])
df=df.dropna()
```

```
X=df[['Size']]
y=df[['Rent']]
scaler=StandardScaler()
X_Scaled=scaler.fit_transform(X)
X_train, X_test, y_train, y_test, = train_test_split(X,y,test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
```

```
y_pred=model.predict(X_test)
print("Mean Squared Error = ", mean_squared_error(y_test, y_pred))
print("R2 Score =",r2_score(y_test,y_pred)*100)
plt.scatter(X_test, y_test, color="blue", label="Actual Rent")
plt.scatter(X_test, y_pred, color="red", label="Predicted Rent")
plt.xlabel("Size (Scaled)")
plt.ylabel("Rent")
plt.legend()
plt.show()
plt.scatter(X_test, y_test, color="blue", label="Actual Rent")
```

LAB – 4 (Logistic Regression)

Loan-Train.csv

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
from sklearn.preprocessing import StandardScaler
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv(r'C:\Users\student\Desktop\loan-train.csv')

df.shape

df.isnull().sum()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col].fillna(df[col].mode()[0], inplace=True)
    else:
        df[col].fillna(df[col].median(), inplace=True)

df_encoded = pd.get_dummies(df, drop_first=True)

df_encoded

df_encoded.columns

df.isnull().sum()

X = df_encoded.drop('Loan_Status_Y', axis=1)
y = df_encoded['Loan_Status_Y']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
model = LogisticRegression(max_iter=1000, solver='lbfgs')
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test_scaled)
```

```
print(f"\nX_test predicted :{y_pred}")
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:\n", cm)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Not Eligible', 'Eligible'],
```

```
            yticklabels=['Not Eligible', 'Eligible'])
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

```
plt.title('Confusion Matrix Loan Eligibility')
```

```
plt.show()
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
print(f"Accuracy = {accuracy:.2f}")
```

```
print(f"Precision = {precision:.2f}")
```

```
print(f"Recall = {recall:.2f}")
```

```
print(f"F1 Score = {f1:.2f}")
```

```
# synthetic_loan_data.csv
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("synthetic_loan_data.csv")
```

```
df.info()
```

```
df.shape
```

```
df.isnull().sum()
```

```
df.columns
```

```
X = df[['Age', 'Income', 'Credit_Score', 'Loan_Amount']]
```

```
y = df['Loan_Eligible']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Not Eligible', 'Eligible'],
```

```
            yticklabels=['Not Eligible', 'Eligible'])
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

```
plt.title('Confusion matrix-Loan Eligibility')
```

```
plt.show()
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
print(f"\n Accuracy = {accuracy:.2f}")
```

```
print(f"\n Precision = {precision:.2f}")
```

```
print(f"\n Recall = {recall:.2f}")
```

```
print(f"\n F1 Score = {f1:.2f}")
```

```
# diabetes.csv
```

```
import numpy as np
```

```
import seaborn as sns
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv('diabetes.csv')

df

df.info()

df.isnull().sum()

df.columns

X = df[['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
        'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']]
y = df['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No', 'Yes'],
            yticklabels=['No', 'Yes'])
```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion matrix-Diabetes')
plt.show()
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print(f"\n Accuracy = {accuracy:.2f}")
print(f"\n Precision = {precision:.2f}")
print(f"\n Recall = {recall:.2f}")
print(f"\n F1 Score = {f1:.2f}")
```

load_diabetes()

```
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

db = load_diabetes()

db.data.shape
```

```
df = pd.DataFrame(db.data, columns=db.feature_names)
```

```
df
```

```
df.info()
```

```
X = df
```

```
y = db.target
```

```
y_binary = (y > np.median(y)).astype(int)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y_binary, test_size=0.3, random_state=42)
```

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['No', 'Yes'],
```

```
            yticklabels=['No', 'Yes'])
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

```
plt.title('Confusion matrix-Diabetes classification')
```

```
plt.show()
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print(f"\n Accuracy = {accuracy:.2f}")
print(f"\n Precision = {precision:.2f}")
print(f"\n Recall = {recall:.2f}")
print(f"\n F1 Score = {f1:.2f}")
```

LAB – 5 (Entropy, Gini, Random Forest)

PlayTennis.csv

```
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.preprocessing import LabelEncoder
from sklearn import tree
from sklearn import metrics

data = pd.read_csv("PlayTennis.csv")

le = LabelEncoder()
for col in data.columns:
    data[col] = le.fit_transform(data[col])

X = data.drop("Play", axis=1)
y = data["Play"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
clf = DecisionTreeClassifier(criterion='entropy')
```

```
clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)
```

```
accuracy = clf.score(X_test, y_test)
```

```
print(f"Accuracy of the model: {accuracy * 100:.2f}%")
```

```
clf = DecisionTreeClassifier(criterion='gini')
```

```
clf.fit(X_train, y_train)
```

```
y_pred = clf.predict(X_test)
```

```
accuracy = metrics.accuracy_score(y_test, y_pred)
```

```
print(f"Accuracy of the model: {accuracy * 100:.2f}%")
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix:\n", cm)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
```

```
disp.plot()
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

```
# diabetes.csv - CART
```

```
import pandas as pd
```

```
from sklearn.tree import plot_tree
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.read_csv("diabetes.csv")

X = data.drop("Outcome", axis=1)
y = data["Outcome"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

id3_model = DecisionTreeClassifier(criterion='entropy')
id3_model.fit(X_train, y_train)

y_pred_id3 = id3_model.predict(X_test)

cm_id3 = confusion_matrix(y_test, y_pred_id3)

sns.heatmap(cm_id3, annot=True, fmt='d', cmap='Blues')
plt.title("ID3 Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

print("ID3 Accuracy:", accuracy_score(y_test, y_pred_id3))
```

```
plt.figure(figsize=(15,10))
```

```
plot_tree(  
    id3_model,  
    feature_names=X.columns,  
    class_names=['No', 'Yes'],  
    filled=True,  
    max_depth=2  
)
```

```
plt.title("ID3 Decision Tree (Depth = 2)")  
plt.show()
```

```
cart_model = DecisionTreeClassifier(criterion='gini')  
cart_model.fit(X_train, y_train)
```

```
y_pred_cart = cart_model.predict(X_test)
```

```
cm_cart = confusion_matrix(y_test, y_pred_cart)
```

```
sns.heatmap(cm_cart, annot=True, fmt='d', cmap='Greens')  
plt.title("CART Confusion Matrix")  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.show()
```

```
print("CART Accuracy:", accuracy_score(y_test, y_pred_cart))
```

```
plt.figure(figsize=(15,10))
```

```
plot_tree(
```

```
    cart_model,  
    feature_names=X.columns,  
    class_names=['No', 'Yes'],  
    filled=True,  
    max_depth=2  
)  
  
plt.title("CART Decision Tree (Depth = 2)")  
plt.show()
```

diabetes.csv - Random forest

```
import pandas as pd  
from sklearn.tree import plot_tree  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import RandomForestClassifier  
from sklearn.metrics import confusion_matrix, accuracy_score  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
data = pd.read_csv("diabetes.csv")  
  
X = data.drop("Outcome", axis=1)  
y = data["Outcome"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42  
)
```

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
y_pred = rf_model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("Confusion Matrix\n", cm)
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
```

```
plt.title("Random Forest Confusion Matrix")
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

```
single_tree = rf_model.estimators_[0]
```

```
plt.figure(figsize=(12,8))
```

```
plot_tree(
```

```
    single_tree,
```

```
    feature_names=X.columns,
```

```
    class_names=['No', 'Yes'],
```

```
    filled=True,
```

```
    max_depth=2
```

```
)
```

```
plt.title("Random Forest Tree (visualized up to depth 2)")
```

```
plt.show()
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f"Accuracy: {accuracy * 100:.2f}%")
```

iris = load_iris() – Random Forest

```
from sklearn.datasets import load_iris
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix\n", cm)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names,
            yticklabels=iris.target_names)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest - Iris Confusion Matrix")
plt.show()
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

Random Forest - Datafram Weather

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt
import seaborn as sns

print("Libraries imported successfully!")

np.random.seed(42)
n_samples = 1000

df = pd.DataFrame({
    'Weather': np.random.choice(['Clear', 'Rainy', 'Foggy'], n_samples),
    'Road_Condition': np.random.choice(['Dry', 'Wet', 'Icy'], n_samples),
    'Time_of_Day': np.random.choice(['Day', 'Night'], n_samples),
    'Traffic_Density': np.random.randint(1, 10, n_samples),
    'Speed': np.random.randint(40, 120, n_samples),
```

```

    'Alcohol': np.random.binomial(1, 0.2, n_samples),
    'Accident': np.random.binomial(1, 0.25, n_samples),
})

print("Shape:", df.shape)
print("\nHead:\n", df.head())

print("\nTarget distribution:\n", df['Accident'].value_counts(normalize=True))

df_encoded = pd.get_dummies(
    df,
    columns=['Weather', 'Road_Condition', 'Time_of_Day'],
    drop_first=True
)

X = df_encoded.drop('Accident', axis=1)
y = df_encoded['Accident']

print("\nAfter encoding - Features shape :", X.shape)
print("Target shape :", y.shape)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

print(f"Train set: {X_train.shape[0]} samples")
print(f"Test set: {X_test.shape[0]} samples")

```

```
dt = DecisionTreeClassifier(  
    criterion='entropy',  
    max_depth=5,  
    random_state=42  
)
```

```
dt.fit(X_train, y_train)
```

```
y_pred_dt = dt.predict(X_test)
```

```
print("\nDecision Tree (ID-3) trained")
```

```
rf = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)
```

```
rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)
```

```
def plot_confusion_matrix(y_true, y_pred, title):
```

```
    cm = confusion_matrix(y_true, y_pred)
```

```
    plt.figure(figsize=(5,4))
```

```
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
```

```
                xticklabels=['No Accident', 'Accident'],
```

```
                yticklabels=['No Accident', 'Accident'])
```

```
    plt.title(title)
```

```
plt.xlabel('Predicted')
```

```
plt.ylabel('Actual')
```

```
plt.show()
```

```
print("Decision Tree Performance")
```

```
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred_dt))
```

```
print("Decision Tree Performance")
```

```
print("Classification Report:")
```

```
print(classification_report(y_test, y_pred_rf))
```

```
plot_confusion_matrix(y_test, y_pred_dt, "Decision Tree Confusion Matrix")
```

```
plot_confusion_matrix(y_test, y_pred_rf, "Random Forest Confusion Matrix")
```

```
from sklearn.tree import plot_tree
```

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(
```

```
    dt,
```

```
    feature_names=X.columns.tolist(),
```

```
    class_names=['No Accident', 'Accident'],
```

```
    filled=True,
```

```
    rounded=True,
```

```
    max_depth=3
```

```
)
```

```
plt.title("Decision Tree (First 3 Levels)")
```

```
plt.show()
```

